Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе битовых полей»

Выполнил:
студент группы 3825Б1ФИип2
Макарычева А.Д.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 5
3. Руководство программиста 6
3.1. Описание структуры программы 6
3.2. Описание структур данных 6-7
3.3. Описание алгоритмов 7-9
4. Результаты экспериментов и тестирования 10
Заключение 11
Литература 12

Введение

Множество — фундаментальное понятие дискретной математики, широко применяемое в программировании для хранения уникальных элементов и выполнения операций объединения, пересечения, проверки принадлежности.

Битовое поле — массив машинных слов, в котором каждый бит хранит состояние одного элемента (1 бит на элемент). Такой подход даёт компактное хранение и высокую скорость теоретико-множественных операций за счёт побитовой обработки слов. Ограничение: битовое представление применимо только к конечному универсуму, а операции над множествами имеют смысл только в рамках одного универсума — при разных мощностях результат теряет смысл.

Цель работы — разработка структуры данных для хранения множеств на основе битовых полей и освоение фреймворка Google Test для автоматического тестирования.

Задачи:

Реализовать класс TBitField — битовое поле с заданным интерфейсом (создание, копирование, установка/сброс/чтение бита, сравнение, побитовые операции |, & — допустимы при разной длине, ~, ввод/вывод).

Реализовать класс TSet — множество на основе битового поля (добавление/удаление/проверка элемента, сравнение, объединение и пересечение только для одного универсума, дополнение, ввод/вывод).

Написать и обеспечить работоспособность тестов Google Test, покрывающих основной функционал и краевые случаи.

1. Постановка задачи

Разработать библиотеку для работы с конечными множествами над универсумом U = {0, 1, …, MaxPower−1} на основе битового поля. Объявления классов в include/tbitfield.h и include/tset.h не изменяются, реализация — в .cpp

TBitField

Поле длиной BitLen бит в массиве слов pMem из MemLen элементов TELEM. Реализовать: конструктор TBitField(int len) (len < 0 → std::invalid_argument), конструктор копирования, деструктор; GetLength(); SetBit / ClrBit / GetBit с проверкой диапазона (std::out_of_range); operator==, operator!=, operator=; operator|, operator& — допустимы при разной BitLen; результат имеет длину max(BitLen, bf.BitLen), недостающие старшие биты меньшего поля считаются нулями (для | — копируются из длинного поля, для & — остаются нулевыми); operator~; operator<< / operator>> — строка из 0/1 длиной BitLen, иначе failbit

TSet

Хранит MaxPower и BitField длины MaxPower. Реализовать: конструкторы TSet(int), копирования, TSet(const TBitField&); operator TBitField(); GetMaxPower(); InsElem / DelElem / IsMember с проверкой 0 ≤ e < MaxPower (std::out_of_range); operator== / operator!= (сначала по MaxPower, затем по полю); operator=; operator+(int) / operator−(int) — добавить/удалить элемент из копии; operator+ / operator* — объединение и пересечение только при MaxPower == s.MaxPower, иначе std::invalid_argument; operator~ — дополнение; operator<<; operator>>

Краевые случаи

len = 0 → MemLen ≥ 1; инверсия с «хвостом» → обнулять биты за пределами BitLen; разные мощности в +, * (TSet) → std::invalid_argument; для TBitField разная длина допустима — результат длины max; некорректный ввод → failbit; динамическая память → копирование, самоприсваивание, освобождение в деструкторе

Тестирование

Набор тестов Google Test: штатное поведение, краевые случаи

2. Руководство пользователя

Данная работа не является консольным приложением с интерактивным вводом. Вся проверка корректности реализации выполняется через автотесты Google Test, которые покрывают основной функционал классов TBitField и TSet, а также краевые случаи

Дополнительно в проекте присутствует файл main.cpp — демонстрационная программа, которая показывает работу с классами (создание поля, установка битов, объединение, пересечение, дополнение множеств, вывод в поток). Она не требует интерактивного ввода и может быть запущена для наглядной проверки результатов

3. Руководство программиста

3.1. Описание структуры программы

Программа состоит из четырёх модулей:
tbitfield.h-объявление TBitField
tset.h-объявление TSet
tbitfield.cpp-реализация TBitField
tset.cpp-реализация TSet
main.cpp-демонстрация
test.cpp-автотесты Google Test

Модуль tbitfield — битовое поле. Содержит тип TELEM, класс TBitField (конструкторы, деструктор, GetLength, SetBit/ClrBit/GetBit, operator==/!=/=, operator|/&/~, вспомогательные GetMemIndex/GetMemMask) и функции operator<</>>

Модуль tset — множество на основе TBitField. Содержит класс TSet (конструкторы, GetMaxPower, InsElem/DelElem/IsMember, operator==/!=/=, operator+/− с элементом и с множеством, operator~, приведение к TBitField) и функции operator<</>>

Модуль main — демонстрационная программа без интерактивного ввода: создаёт поле и множества, показывает все операции, выводит результат

Модуль тестов — набор тестов Google Test для TBitField и TSet, покрывающий штатное поведение и краевые случаи

3.2. Описание структур данных

Класс TBitField

Низкоуровневое битовое поле — динамический массив машинных слов типа TELEM, в котором каждый бит хранит состояние одного элемента

Поля класса
Поле-Тип-Назначение
BitLen-int-длина поля в битах (число хранимых битов)
MemLen-int-количество машинных слов в массиве pMem
pMem-TELEM*-динамический массив слов, хранящий биты

Ключевые инварианты:
MemLen = (BitLen + TELEM_BITS − 1) / TELEM_BITS, минимум 1;
бит n хранится в слове n / TELEM_BITS под маской 1 << (n % TELEM_BITS);
биты за пределами BitLen должны быть нулевыми — иначе ломается сравнение

Класс TSet

Множество над конечным универсумом {0, 1, …, MaxPower−1}, построенное на битовом поле

Поля класса

Поле-Тип-Назначение
MaxPower-int-мощность универсума — количество допустимых элементов
BitField-TBitField-битовое поле длины MaxPower; бит i = 1 ⇔ элемент i принадлежит множеству

Ключевой инвариант:
BitField.GetLength() == MaxPower — длина поля всегда равна мощности универсума.

3.3. Описание алгоритмов

1. Определение индекса слова и маски бита

GetMemIndex(n): n / TELEM_BITS — целочисленное деление даёт номер слова

GetMemMask(n): 1 << (n % TELEM_BITS) — маска с единственной единицей в позиции бита внутри слова

Сложность: O(1)

2. Установка, сброс и чтение бита

SetBit(n): pMem[i] |= mask — побитовое ИЛИ с маской
ClrBit(n): pMem[i] &= ~mask — И с инвертированной маской
GetBit(n): (pMem[i] & mask) ? 1 : 0 — проверка маски
Перед операцией — проверка 0 ≤ n < BitLen, иначе std::out_of_range

Сложность: O(1).

3. Конструктор и деструктор

Конструктор:
Проверить len ≥ 0, иначе std::invalid_argument
Вычислить MemLen по формуле, при необходимости поднять до 1
Выделить new TELEM[MemLen], обнулить все слова
Деструктор: delete[] pMem; pMem = nullptr

4. Копирование и присваивание

Конструктор копирования: выделить новый массив MemLen, поэлементно скопировать слова

operator=:
Проверить самоприсваивание (this == &bf)
Если MemLen различается — освободить старый массив, выделить новый
Скопировать BitLen и все слова

Сложность: O(MemLen)

5. Сравнение

operator==:
Если BitLen различаются — вернуть 0
Иначе поэлементно сравнить слова; при первом несовпадении вернуть 0
Если все совпали — вернуть 1

operator!=: !(*this == bf)

Сложность: O(MemLen)

6. Побитовые операции | и &

Вычислить maxLen = max(BitLen, bf.BitLen) — результат имеет длину большего поля
Создать res(maxLen)
Для общих слов (minMem) применить | (или &)
Для | — скопировать «хвостовые» слова более длинного поля
Для & — «хвостовые» слова остаются нулевыми (недостающие биты = 0)

Сложность: O(MemLen)

7. Инверсия ~

Создать результат той же длины
Поэлементно применить ~ ко всем словам
Если BitLen % TELEM_BITS ≠ 0, обнулить «хвостовые» биты последнего слова маской (1 << tail) – 1
Обрезка хвоста критична: без неё биты за пределами BitLen станут единичными и повлияют на сравнение и вывод

Сложность: O(MemLen)

8. Ввод/вывод TBitField

Вывод: цикл i от BitLen − 1 до 0, вывод GetBit(i). Старший бит — слева
Ввод:
Прочитать строку s
Если |s| ≠ BitLen — установить failbit
Иначе для каждого i от 0 до BitLen − 1 взять символ s[BitLen − 1 − i]; '1' → SetBit(i), '0' → ClrBit(i), иначе — failbit

Сложность: O(BitLen)

9. Конструкторы TSet

TSet(int mp): MaxPower = mp, BitField(mp) — пустое множество
TSet(const TSet&): копирование MaxPower и BitField
TSet(const TBitField& bf): MaxPower = bf.GetLength(), BitField(bf)

10. Операции с элементом

InsElem(e): проверить 0 ≤ e < MaxPower, иначе std::out_of_range; вызвать BitField.SetBit(e)
DelElem(e): проверить диапазон; BitField.ClrBit(e)
IsMember(e): проверить диапазон; вернуть BitField.GetBit(e)
operator+(int e) / operator−(int e): создать копию *this, вызвать InsElem / DelElem на копии, вернуть её. Исходное множество не меняется

Сложность: O(1)

11. Сравнение и присваивание TSet

operator==: если MaxPower различаются — вернуть 0 (без исключения); иначе сравнить BitField

operator!=: !(*this == s)

operator=: проверка самоприсваивания; копирование MaxPower и BitField (внутри — глубокое копирование поля)

12. Объединение и пересечение

operator+ (объединение):

Проверить MaxPower == s.MaxPower, иначе std::invalid_argument

Вычислить bf = BitField | s.BitField

Вернуть TSet(bf)

operator* (пересечение): аналогично, но bf = BitField & s.BitField

Сложность: O(MemLen)

13. Дополнение ~

Вычислить bf = ~BitField (внутри — инверсия с обрезкой хвоста)

Вернуть TSet(bf) — мощность результата равна BitField.GetLength(), то есть MaxPower

Сложность: O(MemLen).

14. Ввод/вывод TSet

Вывод: перебрать i от 0 до MaxPower − 1, для каждого установленного бита вывести индекс через , в формате { … }

Ввод:

Прочитать N. Если N < 0 или ошибка — failbit

Очистить поле (сбросить все биты)

Цикл k от 0 до N − 1: прочитать e; если e ∉ [0, MaxPower) или ошибка — failbit; иначе SetBit(e)

Сложность: O(MaxPower + N)

4. Результаты экспериментов и тестирования

TBitField (12 тестов)

Тест-Назначение

CreateAndLength-проверка создания поля нужной длины и нулевого состояния всех битов

SetAndGetBit-проверка установки и чтения битов, включая первый и последний

ClrBit-проверка сброса ранее установленного бита

OutOfRange-проверка выхода за границы поля (std::out_of_range)

CopyCtorAndAssign-проверка глубокого копирования и присваивания с изменением длины

Equality-проверка == и != для равных, неравных и разной длины полей

Or-проверка побитового ИЛИ двух полей

And-проверка побитового И двух полей

NotWithTail-проверка инверсии с «хвостом» (длина не кратна размеру слова)

OrDifferentLengths- проверка результата длины 10 для | и & при разной длине (5 и 10); биты вычислены с учётом «недостающие = 0»

StreamOut-проверка вывода поля строкой из 0/1

StreamIn-проверка ввода корректной строки и обработки неверной длины (failbit)

TSet (15 тестов)

Тест-Назначение

Empty-проверка создания пустого множества и отсутствия элементов

InsDelMember-проверка добавления, удаления и принадлежности элемента

OutOfUniverse-проверка выхода за универсум (std::out_of_range)

CtorFromBitField-проверка создания множества из битового поля

CastToBitField-проверка приведения множества к битовому полю

Equality-проверка == и != для равных и неравных множеств

Assign-проверка присваивания с изменением мощности

UnionWithElement-проверка добавления элемента копией (a + 4) без изменения исходного

DiffWithElement-проверка удаления элемента копией (a - 4) без изменения исходного

Union-проверка объединения множеств одного универсума

Intersection-проверка пересечения множеств одного универсума

Complement-проверка дополнения множества до универсума

UnionDifferentUniverses-проверка исключения std::invalid_argument для + и * при разной мощности

StreamOut-проверка вывода множества в формате { 1, 4 }

StreamIn-проверка ввода N e1 … eN и обработки недопустимого элемента (failbit)

Все 27 автотестов проходят успешно.

Покрыты все требования задания: функционал классов, краевые случаи, обработка ошибок

Заключение

В ходе работы достигнуты следующие результаты

Реализован класс TBitField — битовое поле на динамическом массиве машинных слов: создание, копирование, деструктор, доступ к битам с контролем диапазона, сравнение, присваивание, побитовые операции |, &, ~ (с обрезкой «хвоста») и ввод/вывод строкой из 0/1

Реализован класс TSet — множество над конечным универсумом на основе TBitField: добавление, удаление и проверка элемента, сравнение, присваивание, операции с элементом, объединение, пересечение, дополнение и ввод/вывод в формате { 1, 4 } / N e1 … eN

Обеспечена корректная работа с памятью — глубокое копирование, самоприсваивание, освобождение в деструкторе

Написаны автотесты Google Test — 27 тестов (12 для TBitField, 15 для TSet), покрывающих штатное поведение, краевые случаи (нулевая длина, инверсия с «хвостом», граничные индексы, разная мощность) и обработку ошибок (std::out_of_range, std::invalid_argument, failbit). Все тесты проходят

Цель работы достигнута

Литература

Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.

Сайт cppreference: https://en.cppreference.com/